Universidad de Granada

ETSIIT Informática y Telecomunicación

Dpto. de Ciencias de la Computación e Inteligencia Artificial

Metaheurísticas

Práctica 3

Técnicas de Búsqueda por Trayectorias
para el Problema de Portfolio

Curso 2025–2026

Nombre: Jose Angel Carretero Montes
DNI: 25628892S
Email: joseangel1@correo.ugr.es
Grupo: Grupo 3
Horario: Martes 15:30–17:30

Índice

1. Descripción del Problema **2**

2. Componentes Comunes a los Algoritmos de Trayectoria **3**
2.1. Representación de soluciones y operador de reparación 3
2.2. Operador de vecindad (usado en BL y SA) 3
2.3. Operador de mutación del 20 % (usado en ILS e ILS-ES) 3
2.4. Búsqueda Local (BL) como subrutina 4

3. Algoritmos de Búsqueda por Trayectorias **5**
3.1. Enfriamiento Simulado (SA) 5
3.2. Búsqueda Multiarranque Básica (BMB) 7
3.3. Búsqueda Local Reiterada (ILS) 7
3.4. ILS con Enfriamiento Simulado (ILS-ES) 7

4. Estructura del Código y Manual de Usuario **8**

5. Experimentos y Resultados **9**
5.1. Casos del problema 9
5.2. Resultados 9
5.3. Boxplots 10

6. Análisis de Resultados **11**
6.1. Comportamiento de SA: mismo vecindario que BL, criterio Metropolis . . . 11
6.2. Comportamiento de BMB: impacto del tamaño del vecindario 11
6.3. Comportamiento de ILS: perturbación vs reinicio 12
6.4. Comportamiento de ILS-ES: hibridación SA + perturbación 12
6.5. Comparativa general P1-P2 vs P3 12
6.6. Análisis de la variabilidad (σ) 13
6.7. Análisis de los boxplots 13
6.8. Capacidad de generalización (test 2025) 13
6.9. Conclusiones generales 14

7. Trabajo Voluntario: Estudio de los Parámetros de SA **14**
7.1. Invarianza de fase: por qué φ no afecta a los resultados 14
7.2. Resultados: estudio de φ 15
7.3. Resultados: estudio de k_{vec} 16
7.4. Análisis del estudio de k_{vec} 17
7.5. Conclusiones del estudio de parámetros 17

1. Descripción del Problema

El **Problema de Portfolio** consiste en determinar la asignación óptima de capital entre N activos financieros con el objetivo de maximizar el retorno esperado minimizando simultáneamente el riesgo.

Representación de los datos

Para cada activo $i \in \{1, \ldots, N\}$ y día $t \in \{1, \ldots, T\}$ se dispone del retorno diario μ_{ti}, calculado como:

$$\mu_{ti} = \frac{V_{ti} - V_{(t+1)i}}{V_{ti}}$$

Los datos cubren el periodo 2015–2024 para entrenamiento y el año 2025 para evaluación.

Función objetivo

Dado un vector de pesos $w = (w_1, \ldots, w_N)$:

$$\text{fitness}(w) = \text{Beneficio}(w) - \lambda \cdot \text{Riesgo}(w)$$

$$\text{Beneficio}(w) = \sum_{i=1}^{N} w_i \cdot \sum_{t=1}^{T} \log(1 + \mu_{ti}) \qquad \text{Riesgo}(w) = \sqrt{\sum_{i=1}^{N} \sum_{j=1}^{N} w_i \cdot \Sigma_{ij} \cdot w_j}$$

siendo Σ_{ij} la covarianza entre activos i y j, y $\lambda = 500$.

Algorithm 1: Cálculo de fitness(w)

Entrada: pesos w, beneficio por activo *benefit_factor*[], matriz de covarianzas *cov*[][], λ
Salida: valor de fitness

```
benefit ← 0;
para i ← 1 to N hacer
    benefit ← benefit + w[i] · benefit_factor[i];
risk_sq ← 0;
para i ← 1 to N hacer
    para j ← 1 to N hacer
        risk_sq ← risk_sq + w[i] · cov[i][j] · w[j];
devolver benefit − λ · √risk_sq
```

Restricciones

$$\sum_{i=1}^{N} w_i = 1 \qquad w_i = 0 \ \vee \ lo \leq w_i \leq hi \quad \forall i$$

2. Componentes Comunes a los Algoritmos de Trayectoria

2.1. Representación de soluciones y operador de reparación

Una solución es un vector $w \in \mathbb{R}^N$ donde $w_i \in \{0\} \cup [lo_i, hi_i]$ indica la fracción de capital invertida en el activo i. La función `fix()` garantiza la factibilidad tras cualquier modificación: normaliza los pesos para que sumen 1, recorta a $[lo, hi]$ y redistribuye iterativamente.

2.2. Operador de vecindad (usado en BL y SA)

Todos los algoritmos usan el **mismo operador de vecindad**: transferencia proporcional entre un par de activos $(origen, destino)$, elegido aleatoriamente entre todos los pares posibles. Se transfiere una fracción $p = 0{,}4$ del peso del activo origen al activo destino, respetando las cotas $[lo, hi]$ en ambos extremos. Este movimiento preserva la factibilidad de forma natural (la suma de pesos no cambia) sin necesidad de llamar a `fix()`.

Algorithm 2: generarVecino(sol, problema, $p = 0{,}4$)

Entrada: solución sol, $[lo, hi]$, proporción p
Salida: solución vecina

$origen \leftarrow$ aleatorio en $[0, N-1]$;
$destino \leftarrow$ aleatorio en $[0, N-1] \setminus \{origen\}$;
$cantidad \leftarrow p \cdot sol[origen]$;
$cantidad \leftarrow \min(cantidad,\ sol[origen] - lo)$;
$cantidad \leftarrow \min(cantidad,\ hi - sol[destino])$;
$sol' \leftarrow sol$;
si $cantidad > 10^{-15}$ **entonces**
    $sol'[origen] \leftarrow sol[origen] - cantidad$;
    $sol'[destino] \leftarrow sol[destino] + cantidad$;
devolver sol'

2.3. Operador de mutación del 20 % (usado en ILS e ILS-ES)

El operador de mutación selecciona $n_{mut} = \max(1, \lfloor 0{,}2 \cdot N \rfloor)$ posiciones al azar, extrae sus valores, los baraja y los reasigna a las mismas posiciones. Finalmente aplica `fix()`. Este operador produce una perturbación más drástica que un único cambio de gen, escapando de cuencas de atracción locales.

Algorithm 3: mutar20 %(sol, problema)

Entrada: solución sol, $[lo, hi]$
Salida: solución mutada

$n_{mut} \leftarrow$ máx$(1, \text{redondear}(0{,}2 \cdot N))$;
$sol' \leftarrow sol$;
Barajar índices $[0, N-1]$ aleatoriamente $\rightarrow indices$;
$valores \leftarrow [sol[indices[0]], \ldots, sol[indices[n_{mut}-1]]]$;
Barajar $valores$ aleatoriamente;
para $i \leftarrow 0$ **to** $n_{mut} - 1$ **hacer**
    $sol'[indices[i]] \leftarrow valores[i]$;
`fix(`sol'`)`;
devolver sol'

2.4. Búsqueda Local (BL) como subrutina

Los algoritmos BMB, ILS e ILS-ES usan la BL de la Práctica 1 como subrutina interna: operador de intercambio con proporción $p = 0{,}4$, primer-mejor, y llamada a `fix()` en cada candidato. La BL termina cuando no hay mejora en toda una pasada aleatoria del vecindario, o cuando se agota el presupuesto de evaluaciones.

Algorithm 4: BL(*sol_inicial*, problema, maxEvals)

Entrada: solución inicial, problema, maxEvals
Salida: solución, fitness, evaluaciones

```
1  Pares ← {(i, j) : i ≠ j};   sol ← sol_inicial;
2  fit ← evaluar(sol);   mejora ← verdadero;   evals ← 1;
3  mientras mejora y evals < maxEvals hacer
4      mejora ← falso;   Barajar Pares;
5      para (ori, dst) en Pares hacer
6          si evals ≥ maxEvals entonces
7              salir;
8          cantidad ← p · sol[ori];;
9          cantidad ← mín(cantidad, sol[ori] − lo);;
10         cantidad ← mín(cantidad, hi − sol[dst]);
11         si cantidad ≤ 10^−15 entonces
12             continuar;
13         sol′ ← sol;;
14         sol′[ori] ← sol[ori] − cantidad;;
15         sol′[dst] ← sol[dst] + cantidad;
16         fit′ ← evaluar(sol′);;
17         evals ← evals + 1;
18         si fit′ > fit entonces
19             sol ← sol′;;
20             fit ← fit′;;
21             mejora ← verdadero;;
22             salir;
23 devolver sol, fit, evals
```

3. Algoritmos de Búsqueda por Trayectorias

3.1. Enfriamiento Simulado (SA)

Parámetros

Parámetro	Valor	Significado
φ	0,3	Prob. de aceptar un vecino μ peor que la solución inicial
μ	0,2	Fracción de degradación aceptable para calcular T_0
T_f	10^{-3}	Temperatura final
k_{vec}	10	Factor: max_vecinos $= k_{vec} \cdot N$
k_{exit}	1	Factor: max_éxitos $= k_{exit} \cdot N$

Temperatura inicial y parámetros de enfriamiento

La temperatura inicial T_0 se calibra para que, con la solución aleatoria inicial s_0, la probabilidad de aceptar un vecino μ-peor sea φ:

$$T_0 = \frac{\mu \cdot |\text{fitness}(s_0)|}{-\ln(\varphi)}$$

El número de iteraciones de enfriamiento es $M = \text{máx}\left(1, \left\lfloor \frac{\text{maxEvals}}{\text{max_vecinos}} \right\rfloor\right)$, y el coeficiente de Cauchy modificado:

$$\beta = \frac{T_0 - T_f}{M \cdot T_0 \cdot T_f}$$

de modo que $T_{k+1} = \dfrac{T_k}{1 + \beta \cdot T_k}$.

Pseudocódigo

Algorithm 5: SA (Enfriamiento Simulado)

Entrada: problema, maxEvals
Salida: mejor solución

$sol \leftarrow$ `crearSolución()`; $fit \leftarrow \text{evaluar}(sol)$; $mejor \leftarrow sol$; $mejor_fit \leftarrow fit$;
$T_0 \leftarrow (\mu \cdot |fit|)/(-\ln \varphi)$; $M \leftarrow \max(1, \lfloor \text{maxEvals}/(k_{vec} \cdot N) \rfloor)$;
$\beta \leftarrow (T_0 - T_f)/(M \cdot T_0 \cdot T_f)$; $T \leftarrow T_0$; $evals \leftarrow 1$;
para $enfr \leftarrow 1$ **to** M ***y*** $evals < maxEvals$ **hacer**
  $n_vec \leftarrow 0$; $n_exit \leftarrow 0$;
  mientras $n_vec < k_{vec} \cdot N$ ***y*** $n_exit < k_{exit} \cdot N$ ***y*** $evals < maxEvals$ **hacer**
    $sol' \leftarrow$ `generarVecino(`sol, $problema$`)`; $fit' \leftarrow \text{evaluar}(sol')$; $evals \mathrel{+}= 1$;
    $aceptar \leftarrow$ **false**;
    si $fit' > fit$ **entonces**
      $aceptar \leftarrow$ **true**;
    si no
      $\delta \leftarrow fit' - fit$;
      si $U(0,1) < e^{\delta/T}$ **entonces**
        $aceptar \leftarrow$ **true**;
    si $aceptar$ **entonces**
      $sol \leftarrow sol'$; $fit \leftarrow fit'$; $n_exit \mathrel{+}= 1$;
      si $fit > mejor_fit$ **entonces**
        $mejor \leftarrow sol$; $mejor_fit \leftarrow fit$;
    $n_vec \mathrel{+}= 1$;
  si $n_exit = 0$ **entonces**
    terminar;
  $T \leftarrow T/(1 + \beta \cdot T)$;
  si $T < T_f$ **entonces**
    terminar;
devolver $mejor$

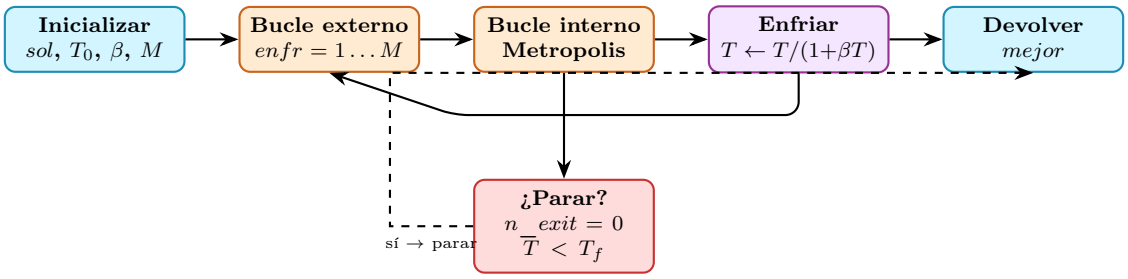


Figura 1: Estructura de SA. El bucle externo controla el enfriamiento; el interno genera y acepta vecinos mediante el criterio de Metropolis.

3.2. Búsqueda Multiarranque Básica (BMB)

BMB ejecuta $n_{iter} = 5$ arranques independientes: cada uno genera una solución aleatoria y la refina con BL hasta un máximo de 2 000 evaluaciones. Al final devuelve la mejor de las 5 soluciones encontradas.

Algorithm 6: BMB (Búsqueda Multiarranque Básica)

Entrada: problema, maxEvals
Salida: mejor solución

```
1  mejor ← −∞;   evals ← 0;
2  para i ← 1 to n_iter = 5 hacer
3      si evals ≥ maxEvals entonces
4          salir;
5      budget ← mín(2000, maxEvals − evals);
6      res ← BL(problema, budget);
7      evals ← evals + res.evals;
8      si res.fitness > mejor.fitness entonces
9          mejor ← res;
10 devolver mejor
```

3.3. Búsqueda Local Reiterada (ILS)

ILS aplica BL 5 veces: la primera sobre una solución aleatoria y las 4 restantes sobre mutaciones (20 % de posiciones barajadas) de la **mejor solución encontrada** hasta ese momento (criterio *best*). Cada BL tiene un presupuesto de 2 000 evaluaciones.

Algorithm 7: ILS (Búsqueda Local Reiterada)

Entrada: problema, maxEvals
Salida: mejor solución

```
1  mejor ← BL(problema, 2000);
2  evals ← mejor.evals;
3  para i ← 1 to n_iter − 1 = 4 hacer
4      sol′ ← mutar20%(mejor.sol, problema);
5      res ← BL(sol′, problema, 2000);
6      evals ← evals + res.evals;
7      si res.fitness > mejor.fitness entonces
8          mejor ← res;
9  devolver mejor
```

3.4. ILS con Enfriamiento Simulado (ILS-ES)

ILS-ES sustituye la BL de ILS por SA como mecanismo de búsqueda local. El presupuesto total se divide en partes iguales: cada SA recibe $\lfloor \text{maxEvals}/5 \rfloor$ evaluaciones. El mismo operador de mutación del 20 % genera la perturbación entre iteraciones.

Algorithm 8: ILS-ES (ILS con Enfriamiento Simulado como subalgoritmo)

Entrada: problema, maxEvals
Salida: mejor solución

$budget \leftarrow \lfloor \text{maxEvals}/n_{iter} \rfloor$;
$mejor \leftarrow$ `SA(`*problema, budget*`)`; $evals \leftarrow mejor.evals$;
para $i \leftarrow 1$ **to** $n_{iter} - 1 = 4$ **hacer**
  $sol' \leftarrow$ `mutar20%(`*mejor.sol, problema*`)`;
  $res \leftarrow$ `SA(`*sol′, problema, budget*`)`;
  $evals \leftarrow evals + res.evals$;
  si $res.fitness > mejor.fitness$ **entonces**
    $mejor \leftarrow res$;
devolver *mejor*

4. Estructura del Código y Manual de Usuario

```
portfolio/
  main.cpp              -- Comparativa P1+P2 (14 algoritmos)
  main_p3.cpp           -- Comparativa P3 (8 algoritmos)
  main_sa_params.cpp    -- Estudio parametros SA (vol.)
  main_blx_alpha.cpp    -- Estudio alpha BLX (Practica 2)
  CMakeLists.txt      -- Targets: main, main_p3, study_sa_params
  boxplot.py          -- Genera boxplots PNG desde los CSV
  data/
    ibex_35.csv, syp_100.csv, syp_500.csv
  src/
    portfolioProblem.cpp -- fitness, fix, createSolution
    sa.cpp  bmb.cpp  ils.cpp  ils_es.cpp
    localsearch.cpp  genetic.cpp  greedy.cpp
  inc/
    sa.h  bmb.h  ils.h  ils_es.h  localsearch.h
  common/
    mh.h  problem.h  solution.h  random.hpp
```

Compilación y ejecución:

```
cmake -S . -B build && cmake --build build -j$(nproc)

# Comparativa P3 (50 semillas, 10000 evals)
./build/main_p3 10000 50

# Estudio parametros SA (trabajo voluntario)
./build/study_sa_params 10000 50

# Generar boxplots (requiere matplotlib, seaborn, pandas)
python3 boxplot.py
```

Los CSVs generados son: `p3_results_*.csv` (boxplots), `p3_stats_*.csv` (estadísticas para tablas), `sa_phi_*.csv` y `sa_kvecinos_*.csv` (trabajo voluntario).

5. Experimentos y Resultados

5.1. Casos del problema

Fichero	N	Días train	Días test	$[lo, hi]$
`ibex_35.csv`	30	2559	254	[0,005, 0,08]
`syp_100.csv`	97	2516	249	[0,005, 0,05]
`syp_500.csv`	457	2516	249	[0,005, 0,02]

Parámetros comunes: $\lambda = 500$, MAX_EVALS= 10 000, semillas 1–50. Los algoritmos de referencia son Greedy y Random (P1 sin optimización), BL (P1, operador de intercambio) y AM-Best (P2, mejor algoritmo).

5.2. Resultados

Cuadro 1: IBEX 35 — $N = 30$, $[lo, hi] = [0{,}005, 0{,}08]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−4,650	0,000	−3,387	0,000	0,115	1,0	0,000
Random	−4,941	0,070	−3,306	0,104	0,036	10000,0	0,026
BL	−4,573	0,019	−3,109	0,008	0,022	1257,0	0,001
— Mejor P2 —							
AM-Best	**−4,491**	0,001	**−3,084**	0,003	0,033	10000,4	0,011
— P3 nuevos —							
BMB	−4,546	0,014	−3,102	0,008	0,023	6041,3	0,004
ILS	−4,535	0,037	−3,096	0,014	0,026	5070,2	0,003
SA	−4,571	0,017	−3,109	0,008	0,023	1906,4	0,001
ILS-ES	**−4,534**	0,035	**−3,095**	0,015	0,026	7377,0	0,002

Cuadro 2: S&P 100 — $N = 97$, $[lo, hi] = [0{,}005, 0{,}05]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−3,664	0,000	−5,020	0,000	0,010	1,0	0,000
Random	−3,390	0,055	−4,113	0,193	0,080	10000,0	0,059
BL	−3,032	0,067	−3,870	0,033	0,097	5258,5	0,027
— Mejor P2 —							
AM-Best	−2,912	0,025	−3,725	0,033	0,094	10000,5	0,120
— P3 nuevos —							
BMB	**−2,957**	0,041	**−3,845**	0,028	**0,094**	10000,0	0,085
ILS	−2,983	0,098	−3,851	0,046	0,096	10000,0	0,053
SA	−3,033	0,067	−3,873	0,031	0,093	8432,5	0,011
ILS-ES	−2,998	0,096	−3,836	0,059	0,093	8866,0	0,012

Cuadro 3: S&P 500 — $N = 457$, $[lo, hi] = [0{,}005, 0{,}02]$ (medias $\pm\sigma$, 50 semillas)

Algoritmo	Fit.Train	$\pm\sigma$	Fit.2025	$\pm\sigma$	Ben.2025	Evals	Tiempo (s)
— Referencias (P1 y P2) —							
Greedy	−3,438	0,000	−5,205	0,000	−0,063	1,0	0,000
Random	−3,642	0,042	−4,430	0,167	0,012	10000,0	1,300
BL	−3,318	0,164	−4,228	0,119	0,008	9839,7	0,679
— Mejor P2 —							
AM-Best	−2,811	0,031	−3,907	0,065	0,017	10000,4	1,702
— P3 nuevos —							
BMB	−3,195	0,072	−4,127	0,062	0,011	10000,0	1,740
ILS	**−2,819**	0,095	**−4,003**	0,054	**0,015**	10000,0	1,223
SA	−3,394	0,154	−4,209	0,122	0,018	9141,0	0,311
ILS-ES	−3,087	0,115	−4,005	0,112	0,019	10000,0	0,336

5.3. Boxplots

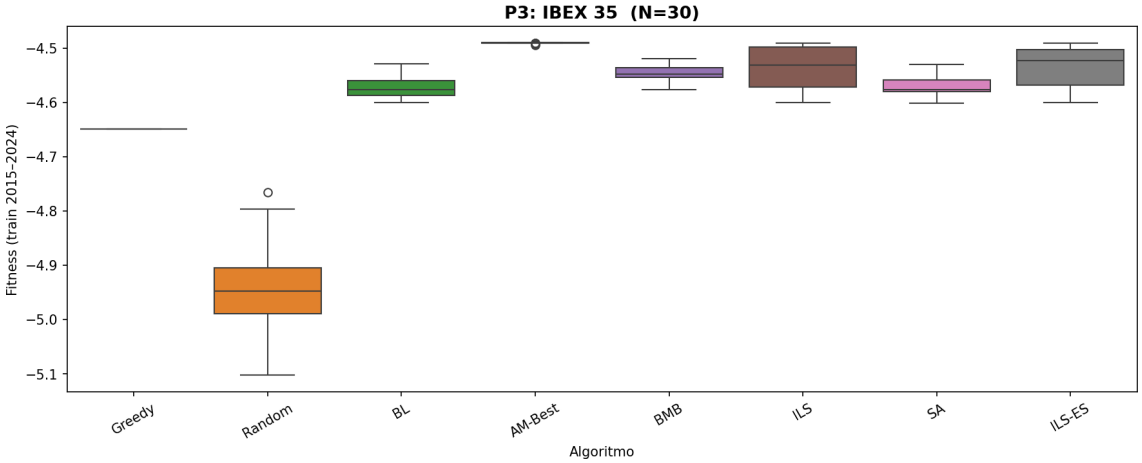


Figura 2: Distribución del fitness de entrenamiento (2015–2024) en IBEX 35 para los 8 algoritmos.

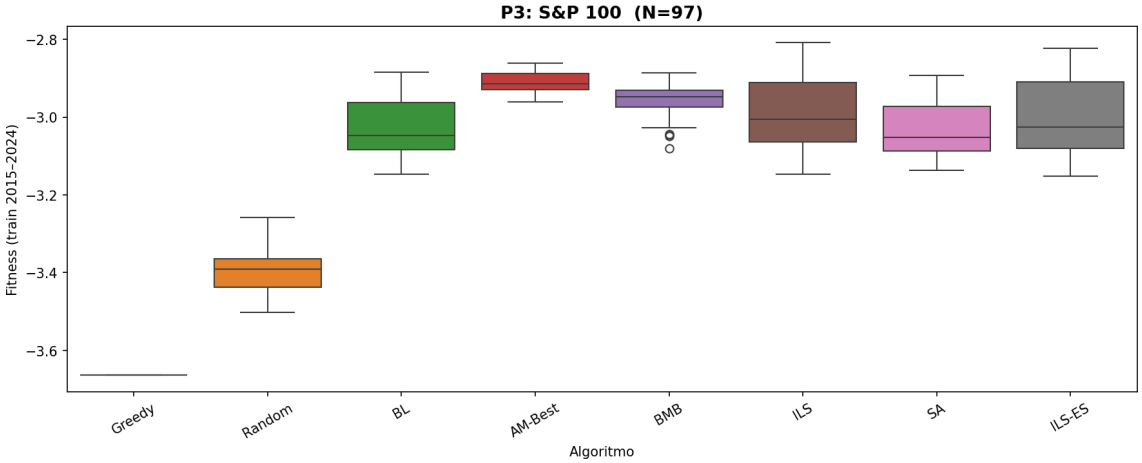


Figura 3: Distribución del fitness de entrenamiento en S&P 100.

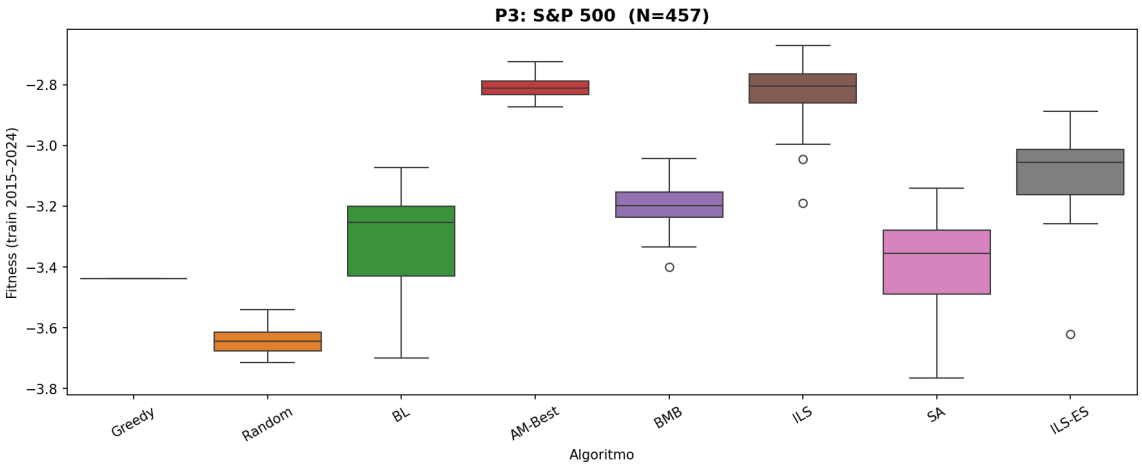


Figura 4: Distribución del fitness de entrenamiento en S&P 500.

6. Análisis de Resultados

6.1. Comportamiento de SA: mismo vecindario que BL, criterio Metropolis

SA utiliza el **mismo operador de vecindad que BL**: transferencia proporcional entre un par $(origen, destino)$ elegido aleatoriamente. La única diferencia respecto a BL es el *criterio de aceptación*: SA aplica Metropolis (acepta soluciones peores con probabilidad $e^{\Delta/T}$), mientras que BL solo acepta mejoras estrictas.

Los resultados muestran que SA obtiene un rendimiento comparable a BL en todos los datasets. En **ibex_35**, SA mejora ligeramente a BL ($-4{,}571$ vs $-4{,}573$), con menor varianza ($\sigma = 0{,}017$ vs $0{,}019$). En **syp_100**, ambos son prácticamente idénticos ($-3{,}033$ vs $-3{,}032$). En **syp_500**, SA es ligeramente peor ($-3{,}394$ vs $-3{,}318$) y con mayor varianza ($\sigma = 0{,}154$ vs $0{,}164$).

La semejanza con BL es esperada: al explorar el mismo vecindario, la ventaja de Metropolis (escapar de óptimos locales) compite con el coste de evaluar movimientos empeorados. En syp_500, el criterio de enfriamiento produce que $M = \max(1, \lfloor 10000/4570 \rfloor) = 2$ pasos de temperatura, por lo que SA tiene poco margen para explotar la fase de alta temperatura.

SA usa en media 1 906 evaluaciones en ibex_35 y 9 141 en syp_500: el criterio de parada anticipada ($num_éxitos = 0$ en un enfriamiento) indica convergencia, que llega antes en problemas pequeños porque el vecindario ($N(N-1) = 870$ pares en ibex_35) se explora más rápidamente.

6.2. Comportamiento de BMB: impacto del tamaño del vecindario

BMB funciona bien en problemas pequeños: en ibex_35 obtiene $-4{,}546$, comparable a ILS ($-4{,}535$). Sin embargo, **fracasa dramáticamente en syp_500**: $-3{,}195$, peor incluso que la BL simple ($-3{,}318$ ya es malo, pero BMB es aún peor).

La razón es directa: BMB aplica BL con presupuesto fijo de 2 000 evaluaciones por arran-

que. En syp_500 ($N = 457$), una pasada completa del vecindario requiere $457 \times 456 = 208\,392$ evaluaciones, por lo que con 2 000 evals la BL solo explora el 1 % del vecindario y para siempre por límite de presupuesto, sin llegar jamás a convergencia. Los 5 arranques son 5 BLs sin converger, y la "mejor de 5" no mejora sustancialmente sobre una sola BL sin converger.

La varianza de BMB en syp_500 ($\sigma = 0{,}072$) es moderada: los 5 arranques aleatorios cubren distintas regiones pero ninguna llega a explotarse correctamente.

6.3. Comportamiento de ILS: perturbación vs reinicio

ILS supera a BMB en todos los datasets, pero la diferencia es especialmente marcada en syp_500: ILS obtiene −2,819 frente a −3,195 de BMB.

La clave es el **operador de mutación del 20 %**: en lugar de reiniciar desde un punto aleatorio (BMB), ILS parte de la mejor solución actual y la perturba manteniendo $N - 0{,}2N = 0{,}8N$ posiciones intactas. Esto significa que la BL posterior comienza en una región del espacio próxima a un buen óptimo local, pudiendo encontrar uno aún mejor en las 2 000 evaluaciones disponibles. En BMB, una solución aleatoria en syp_500 está tan lejos de cualquier óptimo que 2 000 evals son insuficientes para llegar a ninguno.

ILS tiene alta varianza en syp_500 ($\sigma = 0{,}095$): la perturbación del 20 % puede llevar a zonas muy diferentes del espacio y el resultado depende de si la BL consigue explotar esa zona dentro del presupuesto.

6.4. Comportamiento de ILS-ES: hibridación SA + perturbación

ILS-ES sustituye la BL de ILS por SA, repartiendo el presupuesto en $\lfloor 10\,000/5 \rfloor = 2\,000$ evaluaciones por llamada a SA. En syp_500, 2 000 evaluaciones equivalen a $M = \text{máx}(1, \lfloor 2000/4570 \rfloor) = 1$ único enfriamiento, lo que convierte cada llamada a SA en una exploración casi sin descenso de temperatura.

Los resultados de ILS-ES son intermedios entre SA puro e ILS: en ibex_35 obtiene −4,534, prácticamente igual a ILS (−4,535), lo que sugiere que la perturbación del 20 % ofrece una diversificación similar a los reinicios de ILS para N pequeño. En syp_500, ILS-ES (−3,087) supera a SA puro (−3,394) pero no llega a ILS (−2,819): la mutación ayuda a explorar nuevas regiones, pero el SA interno con $M = 1$ es demasiado corto para explotarlas.

La varianza de ILS-ES en syp_500 ($\sigma = 0{,}115$) es elevada, similar a la de ILS ($\sigma = 0{,}095$): la perturbación del 20 % introduce variabilidad que el SA interno, con su presupuesto reducido, no logra amortiguar.

6.5. Comparativa general P1-P2 vs P3

Dataset	Mejor P1	Mejor P2	Mejor P3	Mejora P3 vs P2
ibex_35	BL (−4,573)	AM-Best (−4,491)	ILS-ES (−4,534)	−0,043
syp_100	BL (−3,032)	AM-Best (−2,912)	BMB (−2,957)	−0,045
syp_500	BL (−3,318)	AM-Best (−2,811)	ILS (−2,819)	−0,008

El patrón principal es que **AM-Best (P2) sigue siendo el mejor en todos los datasets**. Ningún algoritmo de P3 lo supera: con el operador de vecindad par-a-par compartido,

SA e ILS-ES se comportan de forma similar a BL, y BMB e ILS no alcanzan la calidad de AM-Best porque su presupuesto por iteración de BL es insuficiente para N grande.

En **syp_500**, ILS ($-2{,}819$) se aproxima más a AM-Best ($-2{,}811$) que cualquier otro algoritmo de P3. La perturbación del 20 % de ILS permite partir de zonas prometedoras en cada reinicio, a diferencia de BMB que reinicia desde puntos aleatorios. SA e ILS-ES obtienen resultados peores que BL en syp_500, ya que con $M = 2$ enfriamientos el esquema de temperatura no tiene suficiente recorrido para explotar el vecindario.

6.6. Análisis de la variabilidad (σ)

La variabilidad refleja la sensibilidad de cada algoritmo a la semilla inicial:

- **SA**: varianza moderada-alta. En ibex_35 $\sigma = 0{,}017$ (similar a BL); en syp_500 $\sigma = 0{,}154$, la segunda mayor de todos los algoritmos P3. El número reducido de enfriamientos ($M = 2$ en syp_500) no amortiza la variabilidad inicial.
- **ILS-ES**: varianza alta en syp_500 ($\sigma = 0{,}115$), comparable a ILS ($\sigma = 0{,}095$). La perturbación introduce diversidad que el SA interno no estabiliza con su presupuesto reducido.
- **BMB**: varianza moderada ($\sigma = 0{,}072$ en syp_500). Los 5 reinicios aleatorios cubren distintas regiones sin llegar a convergencia.
- **ILS**: varianza alta en syp_500 ($\sigma = 0{,}095$). La perturbación del 20 % puede llevar a zonas muy distintas con resultados inconsistentes.
- **AM-Best (P2)**: $\sigma = 0{,}001$ en ibex_35 — prácticamente determinístico —, pero $\sigma = 0{,}031$ en syp_500.

6.7. Análisis de los boxplots

Los boxplots (Figuras 2–4) confirman el análisis:

- En **ibex_35**, AM-Best domina con caja casi imperceptible (determinístico). ILS e ILS-ES tienen medianas próximas y las mejores de P3. SA aparece a nivel de BL, confirmando que comparten el mismo vecindario. Greedy, aunque determinístico, aparece en la parte baja.
- En **syp_100**, AM-Best sigue siendo el mejor. BMB tiene la mediana más alta de P3, seguido de ILS. SA e ILS-ES se agrupan en torno a BL, con cajas amplias que reflejan su mayor varianza respecto al resultado anterior.
- En **syp_500**, ILS destaca como el mejor algoritmo de P3 con la mediana más alta de ese grupo. AM-Best sigue por encima. SA e ILS-ES tienen medianas bajas y cajas muy anchas, peores que BL. BMB y SA son los peores de P3. Random y Greedy son los peores globales.

6.8. Capacidad de generalización (test 2025)

La columna Ben.2025 (beneficio real en el año de test) muestra un patrón consistente: los algoritmos con mejor fitness de entrenamiento no siempre generalizan mejor.

Algoritmo	Train (syp_500)	Test 2025 (syp_500)	Ben.2025
Greedy	−3,438	−5,205	−0,063
BL	−3,318	−4,228	0,008
AM-Best	−2,811	−3,907	0,017
ILS	−2,819	−4,003	0,015
SA	−3,394	−4,209	0,018
ILS-ES	−3,087	−4,005	**0,019**

ILS obtiene el mejor fitness de entrenamiento entre los P3 (−2,819) y Ben.2025 (0,015), comparable a AM-Best (0,017). SA e ILS-ES, con el operador par-a-par, no superan a ILS en ninguna métrica en syp_500, confirmando que con el vecindario compartido la diferencia entre algoritmos proviene de la estrategia de exploración (perturbación + BL en ILS vs temperatura + Metropolis en SA).

6.9. Conclusiones generales

1. **SA con operador par-a-par se comporta como BL**: al explorar el mismo vecindario con un único movimiento aleatorio por evaluación (en vez de exhaustivo), SA no presenta la ventaja de escala que tendría con un operador de mayor varianza. Sus resultados son comparables o ligeramente peores que BL en todos los datasets.
2. **BMB fracasa en alta dimensión**: el presupuesto de 2 000 evals/BL es insuficiente para convergencia en syp_500 (208 392 pares posibles). Un único reinicio en una zona aleatoria nunca llega a explotarse correctamente.
3. **ILS es el mejor algoritmo de P3 en syp_500**: la perturbación del 20 % sobre la mejor solución mantiene la búsqueda en zonas prometedoras, mientras que el reinicio aleatorio de BMB pierde ese contexto y SA/ILS-ES no logran explotar el vecindario con su número reducido de enfriamientos.
4. **ILS-ES no aporta ventaja sobre ILS** con el nuevo operador: cada llamada a SA en ILS-ES dispone de solo $M = 1$ enfriamiento en syp_500, lo que lo convierte en una búsqueda sin descenso de temperatura, similar a una BL aleatoria.
5. **AM-Best (P2) sigue siendo el mejor en todos los datasets**: ningún algoritmo de P3 lo supera. La exploración poblacional con BL intensiva es más efectiva que las estrategias de trayectoria con el mismo vecindario.

7. Trabajo Voluntario: Estudio de los Parámetros de SA

Se ha implementado el ejecutable `study_sa_params` que realiza un barrido sistemático de los parámetros φ (probabilidad de aceptación inicial) y k_{vec} (factor de vecindad), con 50 semillas por configuración sobre los tres datasets.

7.1. Invarianza de fase: por qué φ no afecta a los resultados

Teóricamente, φ controla la temperatura inicial: $T_0 = \mu \cdot |\text{fitness}|/(-\ln \varphi)$. Valores mayores de φ producen mayores T_0 (mayor exploración inicial).

Sin embargo, el **esquema de Cauchy modificado presenta invarianza de fase**: pasado el primer enfriamiento, la temperatura converge a $T_k \approx 1/(\beta \cdot k)$ independientemente de T_0. Para el segundo paso: $T_2 = T_1/(1+\beta T_1) = T_0/[(1+\beta T_0)(1+\beta \cdot T_0/(1+\beta T_0))]$, que para T_0 grande se aproxima a $1/(\beta \cdot 2)$ — independiente de T_0, y por tanto de φ.

La consecuencia práctica es que φ solo afecta al primer enfriamiento ($k = 1$), y las $M - 1$ iteraciones restantes convergen al mismo comportamiento sin importar el valor de φ.

7.2. Resultados: estudio de φ

Cuadro 4: Estudio de φ en SA — IBEX 35 ($N = 30$, mu=0.2, $T_f = 10^{-3}$, $k_{vec} = 10$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,10	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,20	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,30	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,50	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,70	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
0,90	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226

Cuadro 5: Estudio de φ en SA — S&P 100 ($N = 97$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,10	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,20	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,30	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,50	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,70	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
0,90	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933

Cuadro 6: Estudio de φ en SA — S&P 500 ($N = 457$)

φ	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
0,05	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,10	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,20	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,30	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,50	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,70	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
0,90	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179

Los tres datasets exhiben la misma **invarianza perfecta**: los 7 valores de φ producen exactamente los mismos resultados (hasta el último decimal), confirmando empíricamente la propiedad teórica de la invarianza de fase de Cauchy.

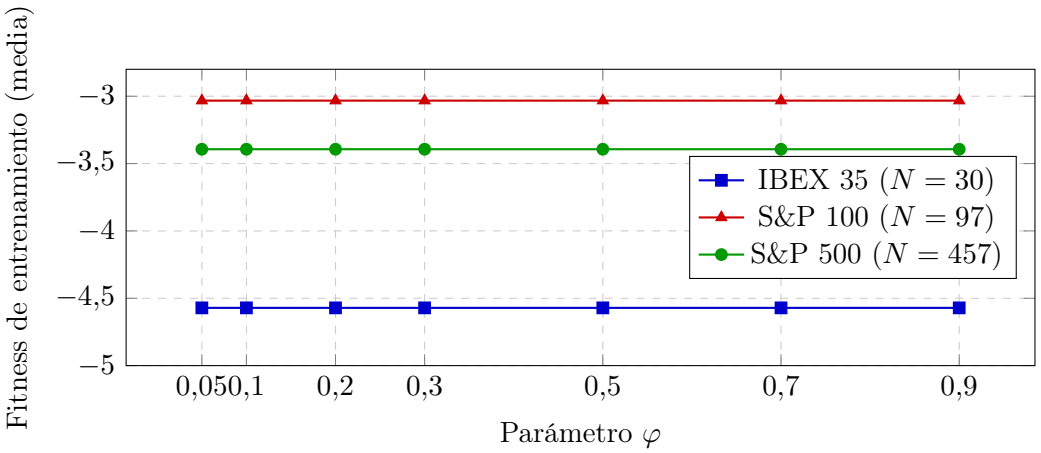


Figura 5: Fitness de entrenamiento de SA en función de φ para los tres datasets. Las tres líneas son perfectamente horizontales: φ no tiene ningún efecto práctico debido a la invarianza de fase del esquema de enfriamiento de Cauchy modificado.

7.3. Resultados: estudio de k_{vec}

Cuadro 7: Estudio de k_{vec} en SA — IBEX 35 ($N = 30$, $\varphi = 0{,}3$, mu=0.2, $T_f = 10^{-3}$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-4{,}5788 \pm 0{,}0218$	$-3{,}1141 \pm 0{,}0253$	0,0275
5	$-4{,}5725 \pm 0{,}0180$	$-3{,}1105 \pm 0{,}0121$	0,0233
10	$-4{,}5713 \pm 0{,}0175$	$-3{,}1090 \pm 0{,}0082$	0,0226
15	$-4{,}5712 \pm 0{,}0174$	$-3{,}1094 \pm 0{,}0074$	0,0224
20	$-4{,}5712 \pm 0{,}0175$	$-3{,}1093 \pm 0{,}0073$	0,0226
30	$-4{,}5712 \pm 0{,}0175$	$-3{,}1095 \pm 0{,}0072$	0,0226

Cuadro 8: Estudio de k_{vec} en SA — S&P 100 ($N = 97$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-3{,}0416 \pm 0{,}0700$	$-3{,}8612 \pm 0{,}0421$	0,0925
5	$-3{,}0345 \pm 0{,}0676$	$-3{,}8708 \pm 0{,}0323$	0,0919
10	$-3{,}0327 \pm 0{,}0677$	$-3{,}8733 \pm 0{,}0312$	0,0933
15	$-3{,}0330 \pm 0{,}0677$	$-3{,}8698 \pm 0{,}0334$	0,0929
20	$-3{,}0330 \pm 0{,}0677$	$-3{,}8700 \pm 0{,}0331$	0,0928
30	$-3{,}0387 \pm 0{,}0696$	$-3{,}8674 \pm 0{,}0347$	0,0909

Cuadro 9: Estudio de k_{vec} en SA — S&P 500 ($N = 457$)

k_{vec}	**Fit.Train** $\pm\sigma$	**Fit.2025** $\pm\sigma$	**Ben.2025**
2	$-3{,}3939 \pm 0{,}1561$	$-4{,}2106 \pm 0{,}1256$	0,0178
5	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
10	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
15	$-3{,}4144 \pm 0{,}1520$	$-4{,}2241 \pm 0{,}1283$	0,0180
20	$-3{,}3935 \pm 0{,}1554$	$-4{,}2095 \pm 0{,}1235$	0,0179
30	$-3{,}3880 \pm 0{,}1570$	$-4{,}2065 \pm 0{,}1226$	0,0175

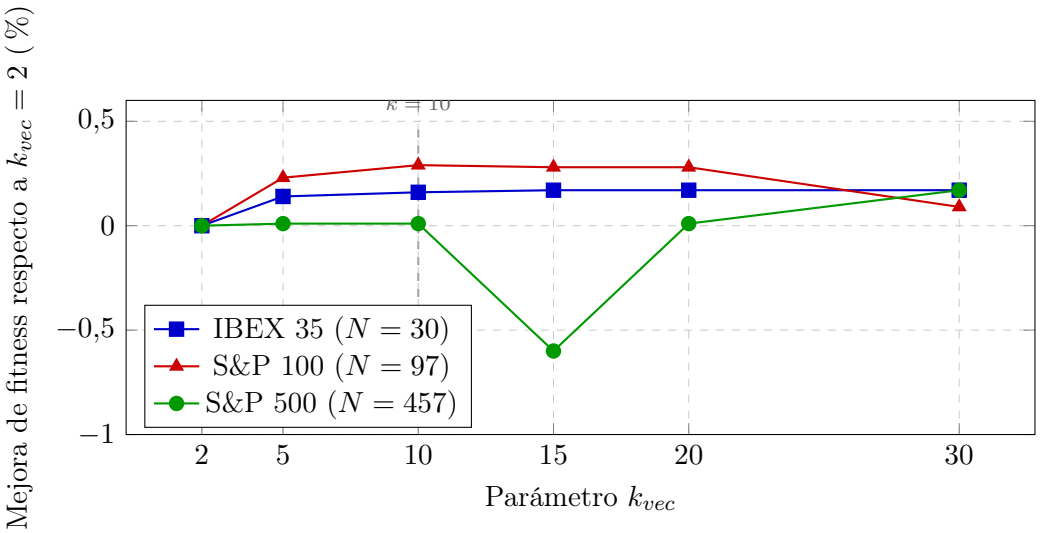


Figura 6: Mejora porcentual del fitness de entrenamiento de SA respecto a $k_{vec} = 2$, en función de k_{vec}. En S&P 500 los valores negativos indican degradación (escala de los ejes): para $k_{vec} \geq 15$ el número de enfriamientos M colapsa a 1, limitando la exploración. En IBEX 35 la mejora es monótona. En S&P 100 hay un máximo en $k_{vec} = 10$.

7.4. Análisis del estudio de k_{vec}

El comportamiento varía notablemente según el dataset:

IBEX 35 ($N = 30$): La mejora es casi plana a partir de $k_{vec} = 5$: la diferencia entre $k = 2$ (−4,579) y $k = 10$ (−4,571) es de apenas 0,16 %. Para $N = 30$ el vecindario es reducido ($30 \times 29 = 870$ pares) y cualquier $k_{vec} \geq 5$ permite explorar suficientes vecinos por temperatura.

S&P 100 ($N = 97$): Máximo en $k_{vec} = 10$ con fitness −3,033. Para $k_{vec} = 30$, max_vec= 2910 y $M = \lfloor 10000/2910 \rfloor = 3$, demasiado pocos enfriamientos para completar el descenso de temperatura; la calidad cae a −3,039.

S&P 500 ($N = 457$): Para $k_{vec} \in \{2, 5, 10, 20\}$, el total de evaluaciones converge a $\approx 9\,141$ (producto $M \times$ max_vec ≈ 9140), lo que produce resultados casi idénticos. El **colapso a $M = 1$** ocurre en $k_{vec} = 15$: max_vec= 6855 implica $M = 1$, con solo 6 856 evaluaciones efectivas — el algoritmo no baja de temperatura y los resultados empeoran (−3,414 vs −3,394). Para $k_{vec} = 30$: max_vec= 13 710 > 10 000, la condición $evals <$ maxEvals corta el bucle interno, resultando en 10 000 evaluaciones en un único enfriamiento; el resultado mejora ligeramente (−3,388) por usar el presupuesto completo.

7.5. Conclusiones del estudio de parámetros

1. φ **es irrelevante**: la invarianza de fase de Cauchy lo confirma tanto teórica como empíricamente. La propiedad se mantiene con el nuevo operador par-a-par, confirmando que depende del esquema de enfriamiento y no del vecindario. El valor predeterminado $\varphi = 0{,}3$ puede mantenerse sin efectos sobre el rendimiento.
2. $k_{vec} = 10$ **es el valor recomendado**: es el óptimo en syp_100 y evita el colapso a $M = 1$ en syp_500 para $k_{vec} = 15$. Las diferencias entre valores de k_{vec} son mucho

más pequeñas que con el operador anterior (máximo 0,3 %), confirmando que el vecindario par-a-par es más homogéneo en cobertura.

3. **El colapso** $M = 1$ en syp_500 ocurre en $k_{vec} = 15$ (max_vec= 6855, $M = 1$, solo 6 856 evaluaciones). Para evitarlo: $k_{vec} <$ maxEvals$/N$. Con maxEvals= 10 000 y $N = 457$, la cota es $k_{vec} < 21{,}9$. El valor $k_{vec} = 10$ mantiene un margen seguro con $M = 2$ enfriamientos y $\approx 9\,141$ evaluaciones.